

EXPERIMENT : 8-Queen problem

Aim

To develop a Python program to solve the 8-Queen problem using the Backtracking algorithm.

Objective

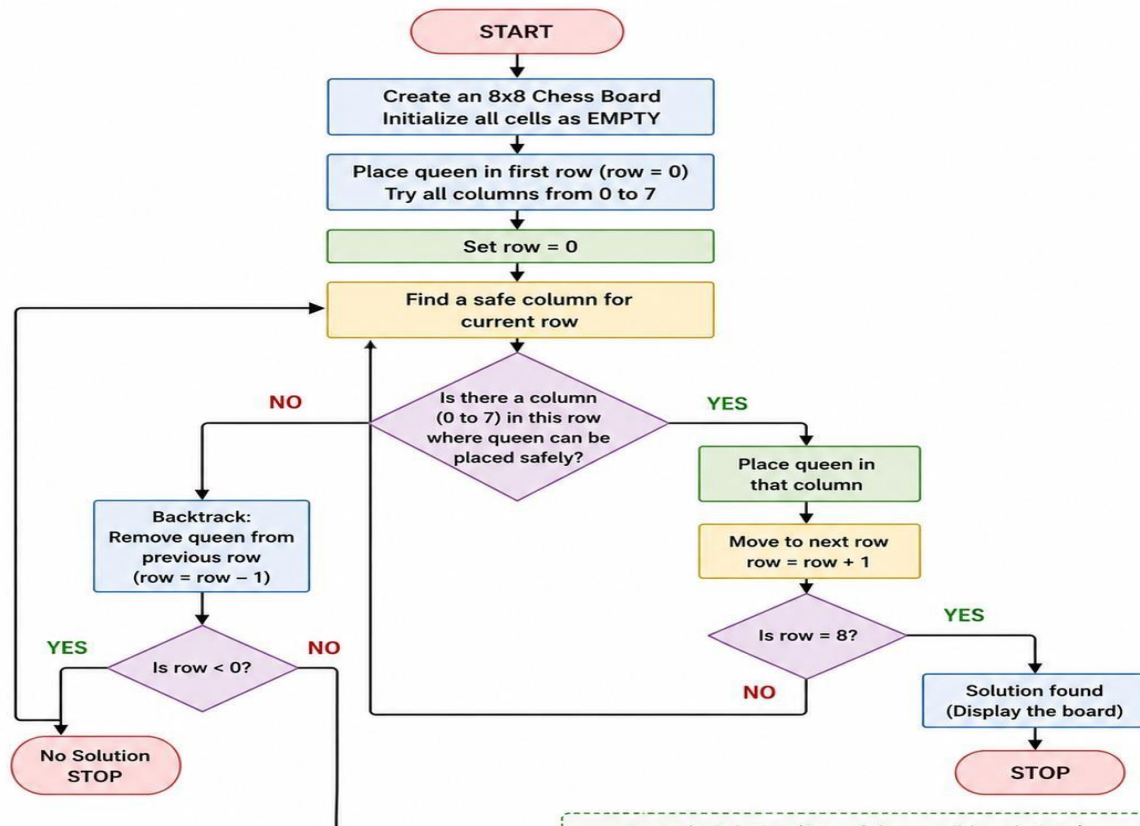
- To understand the Backtracking technique.
- To place 8 queens on an 8×8 chessboard such that no two queens attack each other.
- To display one valid arrangement of the queens.

Algorithm

1. Start.
2. Create an empty 8×8 chessboard.
3. Begin placing queens from the first column.
4. For each row in the current column:
 - Check whether placing a queen is safe.
 - If safe, place the queen.
 - Recursively place queens in the next column.
5. If all 8 queens are placed successfully, print the solution.
6. If no safe position exists, remove the previously placed queen (backtracking) and try another position.
7. If all 8 queens are placed successfully, print the solution.
8. If no safe position exists, remove the previously placed queen (backtracking) and try another position.
9. Repeat until a solution is found or all possibilities are exhausted.
10. Stop.

Flowchart:

8 – QUEEN PROBLEM – FLOWCHART



Safe Position Check

A position (row, col) is safe if no queen exists in:

- Same column
- Same major diagonal (\)
- Same minor diagonal (/)

Example Solution (One of the possible solutions)

	0	1	2	3	4	5	6	7
0		Q						
1					Q			
2								Q
3						Q		
4			Q					
5							Q	
6		Q						
7				Q				

(Q = Queen)

Code:

N = 8

```
def print_board(board):
```

```
    for row in board:
```

```
        print(" ".join("Q" if x else "." for x in row))
```

```
def is_safe(board, row, col):
```

```
    # Check left side
```

```
    for i in range(col):
```

```
        if board[row][i]:
```

```
            return False
```

```

    # Check upper-left diagonal
    i, j = row, col
    while i
    >= 0 and j >= 0:
        if
        board[i][j]:
            return
        False
        i -= 1
        j -= 1

    # Check lower-left diagonal
    i, j = row, col
    while i < N and j >= 0:
    if board[i][j]:
        if
        board[i][j]:
        return False
        i += 1
        j -= 1

    return True

def solve(board, col):
    if col >= N:
        return
    True

    for row in range(N):
        if
        is_safe(board, row, col):
        board[row][col] = 1

        if solve(board, col + 1):
            return True

        board[row][col] = 0

    return False

board = [[0 for _ in range(N)] for _ in range(N)]
if solve(board, 0):
    print("Solution:")
    print_board(board)
else:
    print("No solution exists.")

```

Output



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/sandh/OneDrive/Desktop/EXPERIMENT 6.py
Solution Found:
Q . . . . .
. . . . Q . .
. . . . . Q .
. . Q . . . .
. . . . . Q .
. . . . . Q .
. Q . . . . .
. . . Q . . .
>>>
```

Result

The Python program successfully places all 8 queens on the chessboard such that no two queens attack each other using the Backtracking algorithm.

Conclusion

The 8-Queen problem was solved successfully using the Backtracking algorithm. The algorithm systematically explores possible queen placements and backtracks whenever a conflict occurs, ensuring a valid solution is found. This experiment demonstrates the effectiveness of Backtracking for solving constraint satisfaction problems.